



Tu es un expert en ingénierie logicielle full-stack, architecture applicative,

sécurité, performance et bonnes pratiques de développement.

Je te soumetts le dépôt GitHub suivant : <https://github.com/Terence0-8/SQ-main>

Effectue un audit technique COMPLET et exhaustif de ce repository.

Ne laisse aucun aspect de côté. Voici les dimensions obligatoires à couvrir :

1. ANALYSE STRUCTURELLE

- Architecture globale du projet (monolithe, microservices, MVC, etc.)
- Organisation des dossiers et fichiers (respect des conventions)
- Séparation des responsabilités (frontend / backend / BDD / config)
- Qualité et pertinence du README et de la documentation

2. QUALITÉ DU CODE

- Lisibilité, cohérence du style de code (conventions de nommage, indentation)
- Présence et qualité des commentaires
- Duplication de code (DRY), couplage fort, manque d'abstraction
- Complexité cyclomatique visible (fonctions trop longues, imbrications excessives)
- Utilisation correcte des paradigmes (POO, fonctionnel, etc.)

3. SÉCURITÉ

- Exposition de secrets, tokens, clés API dans le code ou les fichiers de config
- Présence d'un .gitignore adapté
- Gestion des entrées utilisateur (risques XSS, injection SQL, CSRF)
- Authentification et gestion des sessions (si applicable)
- Dépendances présentant des vulnérabilités connues (CVE)
- Politique CORS, headers HTTP de sécurité

4. PERFORMANCE

- Requêtes BDD non optimisées (N+1, absence d'index, over-fetching)
- Chargement inutile de ressources en frontend
- Absence de cache, de lazy loading ou de pagination
- Bundle size, imports inutilisés, tree-shaking

5. DÉPENDANCES & STACK

- Liste des dépendances principales et leur pertinence
- Dépendances obsolètes ou non maintenues
- Versions utilisées vs versions stables actuelles
- Cohérence de la stack technologique

6. TESTS & QUALITÉ

- Présence ou absence de tests (unitaires, intégration, e2e)
- Couverture de test estimée
- CI/CD en place (GitHub Actions, etc.) ou absent
- Linters, formatters, hooks pre-commit

7. ACCESSIBILITÉ & UX (si frontend)

- Respect des standards WCAG de base
- Sémantique HTML
- Responsive design et compatibilité navigateurs

8. DÉPLOIEMENT & DEVOPS

- Configuration de déploiement présente ou absente
- Variables d'environnement correctement gérées
- Dockerisation, containerisation
- Logs et monitoring

SYNTHÈSE & PLAN D'ACTIONS

Après l'audit, génère un plan d'actions structuré en trois horizons :

☒ Court terme (0–2 semaines) — Urgences et quick wins

Liste priorisée des actions critiques (sécurité, bugs bloquants, dette technique majeure).
Pour chaque action : problème identifié → action concrète → fichier/ligne concerné si possible.

☒ Moyen terme (1–3 mois) — Consolidation

Refactoring, ajout de tests, amélioration de l'architecture, mise à jour de dépendances.
Pour chaque action : justification technique + effort estimé (faible / moyen / élevé).

☒ Long terme (3–12 mois) — Vision & scalabilité

Évolutions structurantes, montée en charge, migration technique, documentation complète.
Pour chaque action : impact attendu + prérequis techniques.

FORMAT DE RENDU

- Sois exhaustif : mieux vaut trop que pas assez
- Si une section ne s'applique pas, indique-le explicitement avec la raison
- Conclut avec un score global de maturité technique et UX/UI sur 10 par catégorie

Voici l'audit technique complet et exhaustif du repository [SQ-main](#) ☒

Audit Technique Complet — Solitiquo (SQ-main)

1. Analyse Structurelle

Le projet est un média politique camerounais (Solitiquo) organisé en architecture monolithe hybride : un serveur Node.js/Express sert à la fois les fichiers statiques HTML et une API REST, avec PostgreSQL comme base de données.

Dimension	Observation	Verdict
Architecture	Monolithe Node.js + HTML statique + API REST	⚠ Acceptable en phase MVP, limite la scalabilité
Dossiers racine	backend/, css/, js/, scripts/, uploads/, icons/	⚠ HTML éparpillés à la racine (30+ fichiers)
Séparation front/back	HTML/CSS/JS côté racine, logique métier dans backend/	✗ Pas de séparation claire, couplage fort
README	CHANGELOG.md, DASHBOARD_ANALYTICS_README.md, RECOVERY-REPORT.md présents	⚠ Pas de README principal structuré
Fichiers parasites	migrate_err.txt, test_out.txt, debugUI.js à la racine	✗ Fichiers de debug/artefacts commités

La racine du repo accumule une trentaine de fichiers HTML directement (index.html, admin.html, article.html, auth.html, etc.) sans sous-répertoire pages/ ou views/, ce qui nuit

fortement à la maintenabilité. Le dossier `uploads/` commité dans le repo est une **anomalie critique** — les fichiers uploadés par les utilisateurs ne doivent jamais être versionnés.

2. Qualité du Code

Axe	Constat	Sévérité
Style/nommage	Noms en français (cohérent), mais mélange de conventions camelCase/kebab-case dans les fichiers	☒ Moyen
Commentaires	Présents dans certains fichiers JS, absents dans les HTML de grande taille (60KB pour <code>admin.html</code>)	☒ Moyen
DRY	Header/footer/navigation dupliqués dans chaque HTML (~30 fichiers). Pas de système de composants/templates	☒ Critique
Longueur des fichiers	<code>admin.html</code> = 60 KB, <code>article.html</code> = 43 KB — contiennent du CSS inline et JS inline massifs	☒ Critique
Paradigmes	JavaScript vanilla côté front, Node.js CommonJS côté back ("type": "commonjs")	✓ Cohérent
Fichiers debug	<code>debugUI.js</code> et <code>test_analytics.js</code> commités à la racine	☒ Ne doit pas être en prod

La duplication de la navigation et du footer dans chaque fichier HTML est le problème de qualité le plus coûteux : toute modification graphique implique d'éditer 30+ fichiers manuellement.

3. Sécurité ☒

Risque	Constat	Sévérité
Secrets exposés	<code>.env.example</code> présent ✓, <code>.gitignore</code> présent ✓ — pas de secrets directs détectés	✓ OK
<code>.gitignore</code>	Présent mais le dossier <code>uploads/</code> est commité malgré tout	☒ Critique
Dépendances sécurité	<code>helmet</code> , <code>express-rate-limit</code> , <code>cors</code> , <code>bcrypt</code> , <code> Joi</code> , <code>dompurify</code> présents	✓ Bon outillage
XSS	<code>dompurify</code> côté back mais usage dans JSDOM uniquement — protection XSS côté client non confirmée	☒ À vérifier
Auth	<code>jsonwebtoken</code> + <code>passport</code> (Google, Facebook OAuth) + <code>express-session</code> + <code>connect-pg-simple</code>	⚠ Double système JWT/session à auditer (risque de confusion)
CSRF	Pas de token CSRF visible dans les formulaires HTML	☒ Critique
Multer version	<code>multer@1.4.5-lts.1</code> — version LTS patchée, pas la 2.x qui corrige des failles connues	☒ Moyen
Admin sans auth visible	<code>admin.html</code> (60KB) — vérifier que la route est protégée côté serveur	☒ À confirmer

Risque	Constat	Sévérité
CORS	Package cors présent mais configuration non auditée	☒ À contrôler

La coexistence de JWT et sessions sans architecture claire est un signal d'alerte fort : si mal géré, cela peut créer des failles de contournement d'authentification.

4. Performance ⚡

Point	Constat	Impact
Images à la racine	logo.png (72KB), assemblee.webp (291KB) non optimisées ou servies via CDN	☒ Moyen
Sharp	Package sharp présent — potentiellement utilisé pour le resize, bon signal	✓
Compression	Package compression présent pour gzip Express	✓
Service Worker	sw.js + manifest.json → PWA configurée, offline géré	✓ Bon
CSS/JS inline	Présence massive de styles inline dans les grands HTML (admin.html, article.html)	☒ Pénalise le cache
Pagination	À vérifier dans les routes API (articles, podcasts) — risque over-fetching	☒ À auditer
Cache BDD	Pas de Redis/cache visible — chaque requête tape directement PostgreSQL	☒ Moyen terme
Bundle	Pas de bundler (Webpack/Vite) — JS servi en fichiers bruts, pas de tree-shaking	☒ Moyen

5. Dépendances & Stack

La stack est cohérente et bien choisie pour un média en croissance.

Package	Version déclarée	Dernière stable (mars 2026)	Statut
express	^5.1.0	5.x	✓
pg	^8.16.3	8.x	✓
bcrypt	^6.0.0	6.x	✓
stripe	^20.4.1	20.x	✓
multer	1.4.5-lts.1	2.x disponible	⚠ Vieille branche LTS
helmet	^8.1.0	8.x	✓
dotenv	^17.2.3	17.x	✓
geoip-lite	^2.0.0	2.x	✓
supertest (dev)	^7.2.2	7.x	✓

Un point notable : Stripe ET CinetPay sont probablement utilisés (d'après le fichier CSV du Space), mais seul Stripe apparaît dans package . json — l'intégration CinetPay semble faite côté client uniquement, ce qui est risqué.

6. Tests & Qualité CI/CD

Élément	Constat	Sévérité
Tests unitaires	"test": "echo \"Tests à configurer\" && exit 0" — aucun test fonctionnel	☒ Critique
Supertest	Présent en devDependencies mais non utilisé	☒ Intention sans réalisation
GitHub Actions	Dossier .github/ présent — workflow(s) à confirmer	☒ Potentiellement en place
Linter/Formatter	Aucun ESLint, Prettier ou Husky détecté dans package . json	☒ Problème
Coverage	0% estimé	☒ Critique
test_out.txt	Fichier de sortie de test commité à la racine	✗ À supprimer

L'absence totale de tests automatisés sur un projet avec paiements (Stripe) et authentification OAuth est le risque qualité le plus élevé.

7. Accessibilité & UX

Critère	Constat	Sévérité
Sémantique HTML	Fichiers HTML volumineux — risque de <div> soup, à vérifier précisément dans les pages clés	☒
Responsive	PWA configurée avec manifest.json, fort signal de responsive design	✓
Compatibilité	HTML5 vanilla, navigateurs modernes couverts	✓
Alt images	Non auditable sans lire chaque HTML, mais les 30+ fichiers rendent l'audit manuel difficile	☒ À vérifier
Contraste/WCAG	Non auditable sans rendu visuel, mais l'absence de linter CSS/HTML accessibility ne garantit rien	☒
Multilingue	FR/EN prévu (d'après le plan de développement) — non encore implémenté visiblement	☒

8. Déploiement & DevOps

Élément	Constat	Sévérité
Variables d'env	.env . example présent et structuré ✓	✓
Docker	Aucun Dockerfile ni docker-compose . yml détecté	☒ Manque

Élément	Constat	Sévérité
CI/CD	<code>.github/</code> présent — contenu à confirmer	☒
Migrations BDD	<code>node-pg-migrate</code> configuré avec scripts <code>migrate:up/down/status</code>	✔ Bon
Logs	Aucun système de logging structuré visible (Winston, Pino)	☒ Manque
Monitoring	Pas de Sentry, Datadog ou équivalent détecté	☒ Manque
<code>uploads/</code> versionné	Dossier de fichiers utilisateurs commité dans Git	☒ Critique

☒ Plan d'Actions Structuré

☒ Court terme (0–2 semaines) — Urgences

#	Problème identifié	Action concrète	Fichier concerné
1	Dossier <code>uploads/</code> versionné dans Git	Ajouter <code>uploads/</code> au <code>.gitignore</code> , purger l'historique avec <code>git filter-branch</code> ou BFG	<code>.gitignore</code>
2	Fichiers parasites commités	Supprimer <code>debugUI.js</code> , <code>test_out.txt</code> , <code>migrate_err.txt</code> , <code>test_analytics.js</code> de la racine	Racine du repo
3	Absence de protection CSRF	Implémenter <code>csurf</code> ou tokens CSRF manuels sur tous les formulaires POST	<code>server.js</code> , tous les <code>.html</code> avec <code><form></code>
4	Double système auth JWT + Session confus	Auditer et unifier : choisir JWT stateless OU sessions, pas les deux sans raison	<code>server.js</code> , <code>backend/</code>
5	<code>Admin.html</code> non protégé côté serveur (à vérifier)	S'assurer que la route <code>/admin.html</code> est middleware-protégée par rôle admin	<code>server.js</code>
6	Multer version 1.4.5-lts.1	Migrer vers <code>multer@2.x</code> qui corrige des failles de gestion des fichiers	<code>package.json</code>
7	Aucun linter	Installer ESLint + Prettier + Husky pre-commit hook	<code>package.json</code> , <code>.eslintrc</code>

☒ Moyen terme (1–3 mois) — Consolidation

#	Action	Justification technique	Effort
1	Extraire header/footer/nav dans des composants partagés	Éliminer la duplication dans 30+ HTML, via un moteur de templates (Nunjucks/EJS) ou un build step	Élevé
2	Écrire des tests avec Supertest	Supertest est déjà installé — couvrir les routes critiques (auth, paiement, admin)	Moyen
3	Implémenter un système de logging structuré	Ajouter Winston ou Pino pour avoir des logs exploitables en production	Faible
4	Dockeriser l'application	Créer <code>Dockerfile</code> + <code>docker-compose.yml</code> (app + PostgreSQL) pour déploiement reproductible	Moyen

#	Action	Justification technique	Effort
5	Externaliser les CSS inline	Déplacer les styles des grands HTML vers les fichiers dans /css/ pour activer le cache navigateur	Moyen
6	Sécuriser l'intégration CinetPay côté backend	Si les appels CinetPay sont faits côté client, migrer la logique vers le backend Node.js	Élevé
7	Configurer GitHub Actions CI	Lancer les tests + linter à chaque push/PR	Faible

☒ Long terme (3–12 mois) — Vision & Scalabilité

#	Action	Impact attendu	Prérequis
1	Migrer le frontend vers React/Next.js ou Vue/Nuxt	Composants réutilisables, SSR pour SEO, fin de la duplication HTML	Maîtrise du framework choisi
2	Implémenter un cache Redis	Réduire la charge PostgreSQL sur les articles/podcasts les plus consultés	Infra Redis (Redis Cloud ou Upstash)
3	Séparation en API REST + Frontend découplé	Architecture plus scalable, facilite les applications mobiles futures	Migration frontend + reverse proxy
4	Monitoring & alerting (Sentry + métriques)	Détecter les erreurs en production avant les utilisateurs	Compte Sentry, instrumentation du code
5	Internationalisation complète FR/EN	Ouverture à une audience anglophone, ROI publicitaire amélioré	Bibliothèque i18n (i18next)
6	CDN pour les assets media	Les images/webp/podcasts servis via Cloudinary CDN (déjà installé!) plutôt que le serveur Node.js	Configuration Cloudinary déjà en place

☒ Score de Maturité Technique

Catégorie	Score /10	Commentaire
Architecture	4/10	Monolithe fonctionnel mais structure racine chaotique
Qualité du code	4/10	Duplication massive HTML, fichiers géants
Sécurité	5/10	Bon outillage (helmet, joi, bcrypt) mais CSRF absent, uploads versionnés
Performance	6/10	PWA + compression présents, mais pas de cache BDD ni bundler
Dépendances/Stack	7/10	Stack moderne et cohérente, versions à jour sauf multer
Tests & CI/CD	1/10	Quasi-absence totale de tests — risque majeur sur un projet avec paiements
Accessibilité/UX	5/10	PWA bonne base, mais audit WCAG non réalisable sans rendu
DevOps/Déploiement	4/10	Migrations bien gérées, mais pas de Docker, logging, ni monitoring
Moyenne globale	4.5/10	Projet en phase de construction avancée, fondations partiellement saines

Le projet a de vraies **bonnes intentions techniques** (helmet, joi, passport OAuth, PWA, migrations BDD structurées) mais souffre d'une **dette technique de croissance rapide** typique des projets MVP qui ont grossi sans refactoring. Les priorités absolues sont : la sécurité CSRF, l'élimination des uploads versionnés, et la mise en place d'au moins un minimum de tests sur les routes de paiement.☒

*
**

1. Phase-Objectif-TempsTotal-Cumul.csv
2. Audit complet SQ.pdf